TECHNICAL DOCUMENTATION

AI Dev Team Platform

Руководство по улучшению системы до 10/10

7.5 → 10.0

Версия 1.0 | 2026-03-12

MiniMax Agent

Содержание

1. Обзор текущего состояния
    1.1 Архитектура системы
    1.2 Текущая оценка
    1.3 Найденные проблемы
2. Критические исправления безопасности
    2.1 Миграция секретов
    2.2 Rate Limiting
3. Исправление deprecated API
    3.1 datetime.utcnow()
    3.2 @app.on_event → lifespan
4. Observability и мониторинг
5. Оптимизация базы данных
6. Тестирование и CI/CD
7. Архитектурные улучшения
8. DevOps и инфраструктура
9. Чеклист готовности

1. Обзор текущего состояния

1.1 Архитектура системы

AI Dev Team Platform представляет собой платформу автоматизации разработки на базе 12 микросервисов:

```
┌──────────────────────────────────────────────────────────────┐
│                        NGINX (Reverse Proxy)                 │
└──────────────────────────────────────────────────────────────┘
                              │
          ┌──────────────────┼──────────────────┐
          ▼                  ▼                  ▼
   ┌──────────────┐    ┌──────────────┐    ┌──────────────┐
   │  Web UI      │    │  Chat API    │    │ Orchestrator │
   │  (Frontend)  │    │              │    │     API      │
   └──────────────┘    └──────────────┘    └──────────────┘
                              │                  │
                    ┌─────────┴──────────────────┴──────────┐
                    ▼                                       ▼
              ┌──────────────┐                        ┌──────────────┐
              │ Task Runner  │                        │   Planner    │
              │              │                        │   Worker     │
              └──────────────┘                        └──────────────┘
                    │
      ┌─────────────┴──────────┬──────────────┬──────────────┐
      ▼                        ▼              ▼              ▼
  ┌──────────┐            ┌──────────┐    ┌──────────┐    ┌──────────┐
  │ Coding   │            │   QA     │    │  SSH     │    │ Browser  │
  │ Worker   │            │ Worker   │    │ Worker   │    │ Worker   │
  └──────────┘            └──────────┘    └──────────┘    └──────────┘

      ┌──────────────────────────────────────────────┐
      │     PostgreSQL  |  Redis  |  MinIO           │
      └──────────────────────────────────────────────┘
```

1.2 Текущая оценка: 7.5/10

Категория	Текущий балл	Целевой балл	Gap
Безопасность	6/10	10/10	-4
Качество кода	8/10	10/10	-2
Архитектура	8/10	10/10	-2
Observability	5/10	10/10	-5

DevOps	7/10	10/10	-3
Тестирование	6/10	10/10	-4

1.3 Найденные проблемы

Файл	Проблема	Строки
`libs/shared/dto.py`	datetime.utcnow	72, 176, 190
`libs/shared/models.py`	datetime.utcnow	78, 105, 106, 166, 192, 211, 215, 235
`apps/memory-service/app/main.py`	@app.on_event	246
`apps/chat-api/app/main.py`	@app.on_event	165, 176
`apps/approval-service/app/main.py`	@app.on_event	12
`apps/task-runner/app/main.py`	@app.on_event	569
`apps/orchestrator-api/app/main.py`	@app.on_event	33
`apps/planner-worker/app/main.py`	@app.on_event	54
`.env`	Plaintext secrets	Весь файл

2. Критические исправления безопасности

КРИТИЧНО: Все API ключи хранятся в plaintext в файле .env. Требуется немедленная миграция.

2.1 Миграция секретов из .env в Docker Secrets

Шаг 1: Создать директорию секретов

```
mkdir -p /root/ai-dev-team-platform/secrets
echo "sk-proj-xxx" > secrets/openai_api_key
echo "sk-ant-xxx" > secrets/anthropic_api_key
chmod 600 secrets/*
```

Шаг 2: Обновить docker-compose.yml

```
secrets:
  openai_api_key:
    file: ./secrets/openai_api_key
  anthropic_api_key:
    file: ./secrets/anthropic_api_key

services:
  orchestrator-api:
    secrets:
      - openai_api_key
      - anthropic_api_key
    environment:
      OPENAI_API_KEY_FILE: /run/secrets/openai_api_key
```

Шаг 3: Код для чтения секретов

```
# libs/shared/config.py

from pathlib import Path
from functools import lru_cache

def read_secret(secret_name: str, env_var: str = None) -> str:
    """Читает секрет из Docker Secrets или env variable"""
    secret_path = Path(f"/run/secrets/{secret_name}")
    if secret_path.exists():
        return secret_path.read_text().strip()
    return os.getenv(env_var or secret_name.upper(), "")

@lru_cache
def get_openai_api_key() -> str:
    return read_secret("openai_api_key", "OPENAI_API_KEY")
```

2.2 Rate Limiting

```
# pip install slowapi

from slowapi import Limiter
from slowapi.util import get_remote_address

limiter = Limiter(key_func=get_remote_address)

@app.post("/api/chat")
@limiter.limit("20/minute")
async def chat(request: Request):
    ...
```

3. Исправление deprecated API

3.1 datetime.utcnow() → datetime.now(timezone.utc)

Проблема: `datetime.utcnow()` deprecated в Python 3.12+

Исправление для models.py

```
# БЫЛО:
from datetime import datetime
created_at = mapped_column(DateTime, default=datetime.utcnow)

# СТАЛО:
from datetime import datetime, timezone

def utc_now():
    return datetime.now(timezone.utc)

created_at = mapped_column(DateTime(timezone=True), default=utc_now)
```

Скрипт автоматического исправления

3. Исправление deprecated API

```
#!/usr/bin/env python3
import re
from pathlib import Path

def fix_file(filepath: str):
    path = Path(filepath)
    content = path.read_text()

    if 'from datetime import' in content and 'timezone' not in content:
        content = re.sub(
            r'from datetime import datetime',
            'from datetime import datetime, timezone',
            content
        )

    content = re.sub(
        r'default=datetime\.utcnow(?!\()',
        'default=lambda: datetime.now(timezone.utc)',
        content
    )

    path.write_text(content)
    print(f"Fixed: {filepath}")

fix_file('libs/shared/dto.py')
fix_file('libs/shared/models.py')
```

3.2 @app.on_event('startup') → lifespan

Проблема: `@app.on_event` deprecated в FastAPI 0.100+

Паттерн миграции

```
# БЫЛО (deprecated):
app = FastAPI()

@app.on_event("startup")
async def startup_event():
    await init_database()

# СТАЛО (современный подход):
from contextlib import asynccontextmanager

@asynccontextmanager
async def lifespan(app: FastAPI):
    # STARTUP
    await init_database()
    await connect_redis()
    yield
    # SHUTDOWN
    await close_database()

app = FastAPI(lifespan=lifespan)
```

4. Observability и мониторинг

4.1 Structured Logging (JSON)

```
# pip install structlog

import structlog

def setup_logging(service_name: str):
    structlog.configure(
        processors=[
            structlog.processors.TimeStamper(fmt="iso"),
            structlog.processors.add_log_level,
            structlog.processors.JSONRenderer()
        ],
    )

logger = get_logger()
logger.info("Task created", task_id="123", priority="high")
```

4.2 Prometheus Metrics

```
# pip install prometheus-fastapi-instrumentator

from prometheus_client import Counter, Histogram

llm_requests_total = Counter(
    'llm_requests_total',
    'Total LLM API requests',
    ['provider', 'model', 'status']
)

llm_cost_total = Counter(
    'llm_cost_dollars_total',
    'Total cost in dollars',
    ['provider', 'model']
)
```

4.3 Sentry Error Tracking

```
# pip install sentry-sdk[fastapi]

import sentry_sdk
sentry_sdk.init(
    dsn=os.getenv("SENTRY_DSN"),
    environment="production",
    traces_sample_rate=0.1,
)
```

5. Оптимизация базы данных

5.1 SQL скрипт индексов

```
-- TASKS
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_tasks_status ON tasks(status);
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_tasks_project_status
ON tasks(project_id, status);

-- TASK_STEPS
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_task_steps_task_id
ON task_steps(task_id);
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_task_steps_task_order
ON task_steps(task_id, step_order);

-- AGENT_RUNS
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_agent_runs_step_id
ON agent_runs(step_id);

-- APPROVALS
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_approvals_pending
ON approvals(status) WHERE status = 'pending';
```

5.2 Connection Pooling (PgBouncer)

```
# docker-compose.yml
services:
  pgbouncer:
    image: edoburu/pgbouncer:1.21.0
    environment:
      POOL_MODE: transaction
      MAX_CLIENT_CONN: 1000
      DEFAULT_POOL_SIZE: 50
```

6. Тестирование и CI/CD

6.1 Структура тестов

```
tests/
├── conftest.py
├── unit/
│   ├── test_model_router.py
│   ├── test_security_utils.py
│   └── test_self_healing.py
├── integration/
│   ├── test_orchestrator_api.py
│   └── test_database.py
└── e2e/
    └── test_full_workflow.py
```

6.2 Пример теста

```
import pytest
from libs.shared.security_utils import validate_path

class TestSecurityUtils:
    @pytest.mark.parametrize("path,expected", [
        ("/workspace/code/main.py", True),
        ("../../../etc/passwd", False),
    ])
    def test_validate_path(self, path, expected):
        assert validate_path(path, base_dir="/workspace") == expected
```

6.3 GitHub Actions

```yaml
name: CI/CD Pipeline
on:
  push:
    branches: [main]

jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: pip install ruff mypy black
      - run: ruff check . && black --check .

  test:
    needs: lint
    runs-on: ubuntu-latest
    steps:
      - run: pytest --cov=libs --cov-report=xml
```

7. Архитектурные улучшения

7.1 Message Queue (RabbitMQ)

```python
import aio_pika

class MessageBroker:
    async def publish(self, queue: str, message: dict):
        await self.channel.default_exchange.publish(
            aio_pika.Message(body=json.dumps(message).encode()),
            routing_key=queue,
        )
```

7.2 Caching Layer

```
def cached(prefix: str, ttl: int = 86400):
    def decorator(func):
        async def wrapper(*args, **kwargs):
            key = f"{prefix}:{hash(str(args))}"
            cached = await cache.get(key)
            if cached:
                return cached
            result = await func(*args, **kwargs)
            await cache.set(key, result, ttl)
            return result
        return wrapper
    return decorator
```

7.3 API Versioning

```
app.include_router(router_v1, prefix="/api/v1")
app.include_router(router_v2, prefix="/api/v2")
```

8. DevOps и инфраструктура

8.1 Secure Dockerfile

```
FROM python:3.12-slim AS builder
WORKDIR /build
COPY requirements.txt .
RUN pip wheel --wheel-dir /wheels -r requirements.txt

FROM python:3.12-slim
RUN groupadd -r app && useradd -r -g app app
COPY --from=builder /wheels /wheels
RUN pip install /wheels/*
USER app
HEALTHCHECK CMD curl -f http://localhost:8000/health
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0"]
```

8.2 Kubernetes Helm

```
services:
  orchestratorApi:
    replicas: 2
    resources:
      requests: {memory: "256Mi", cpu: "100m"}
      limits: {memory: "512Mi", cpu: "500m"}
    autoscaling:
      enabled: true
      minReplicas: 2
      maxReplicas: 10
```

9. Чеклист готовности

Production Readiness Checklist

Безопасность

[ ]Секреты в Vault/Docker Secrets

[ ]Rate limiting включён

[ ]HTTPS везде

[ ]Container security scan

Качество кода

[ ]datetime.utcnow исправлен

[ ]@app.on_event → lifespan

[ ]Type hints 100%

[ ]Тесты >80% coverage

Observability

[ ]JSON logging

[ ]Prometheus metrics

[ ]Grafana dashboards

[ ]Sentry интегрирован

База данных

[ ]Индексы созданы

[ ]Connection pooling

[ ]Бэкапы автоматизированы

CI/CD

[ ]GitHub Actions настроен

[ ]Pre-commit hooks

[ ]Feature flags

Приоритетный план выполнения

Неделя	Задачи	Приоритет
1-2	Секреты, Rate limiting, Deprecated fixes	Критично
3-4	Prometheus, Sentry, JSON logging	Важно
5-6	Тесты, CI/CD, Pre-commit	Важно
7-8	Message Queue, Caching, API versioning	Средне
9+	Kubernetes, DR planning	Средне

Полезные команды

```
# Поиск deprecated кода
grep -rn "datetime.utcnow" --include="*.py" .
grep -rn "@app.on_event" --include="*.py" .

# Тесты
pytest --cov=libs --cov-report=html

# Линтинг
ruff check . --fix && black . && mypy libs/shared

# Docker
docker-compose build --parallel

# БД миграции
alembic upgrade head
```

Документ подготовлен: MiniMax Agent

Дата: 2026-03-12

Версия: 1.0